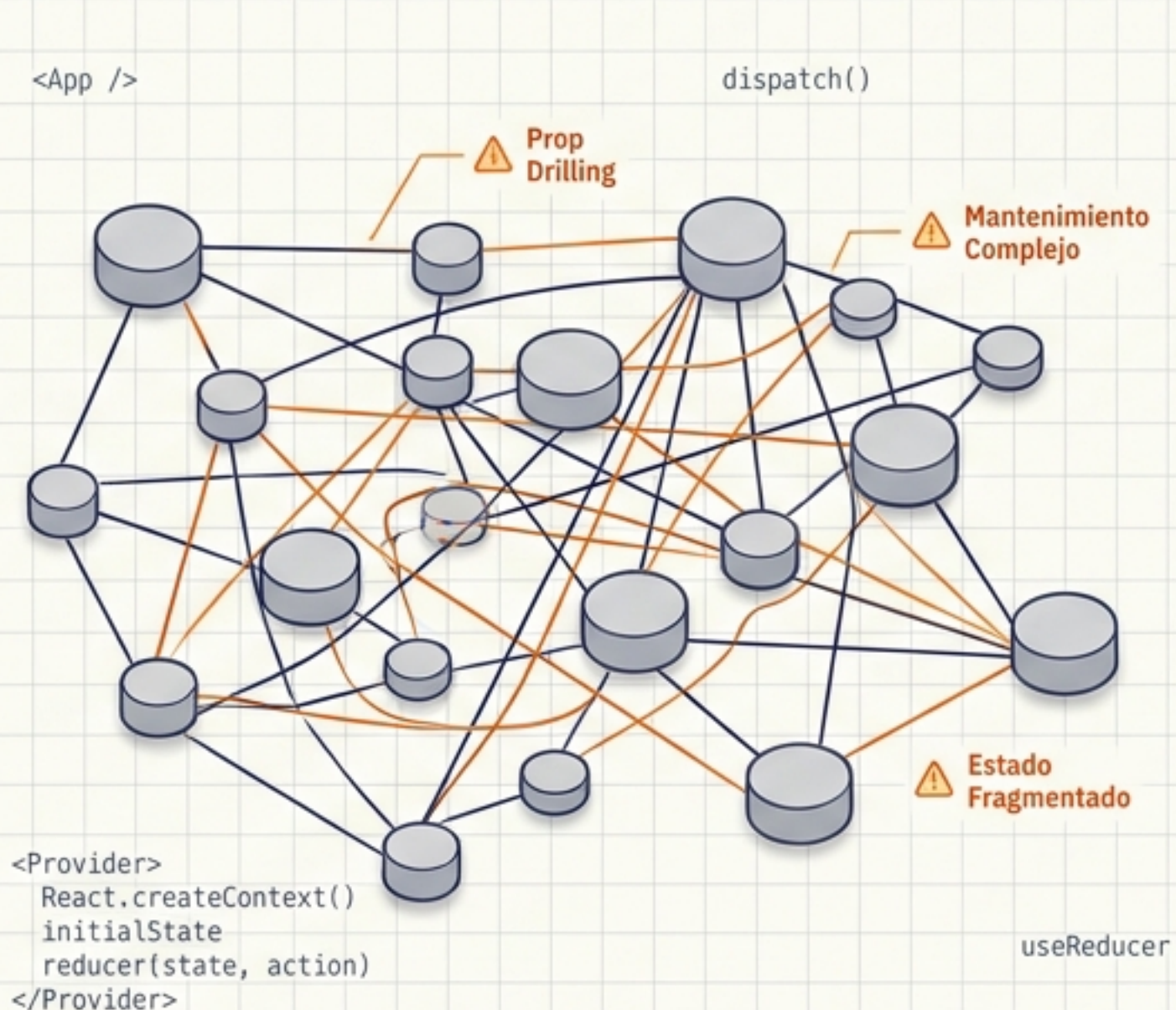
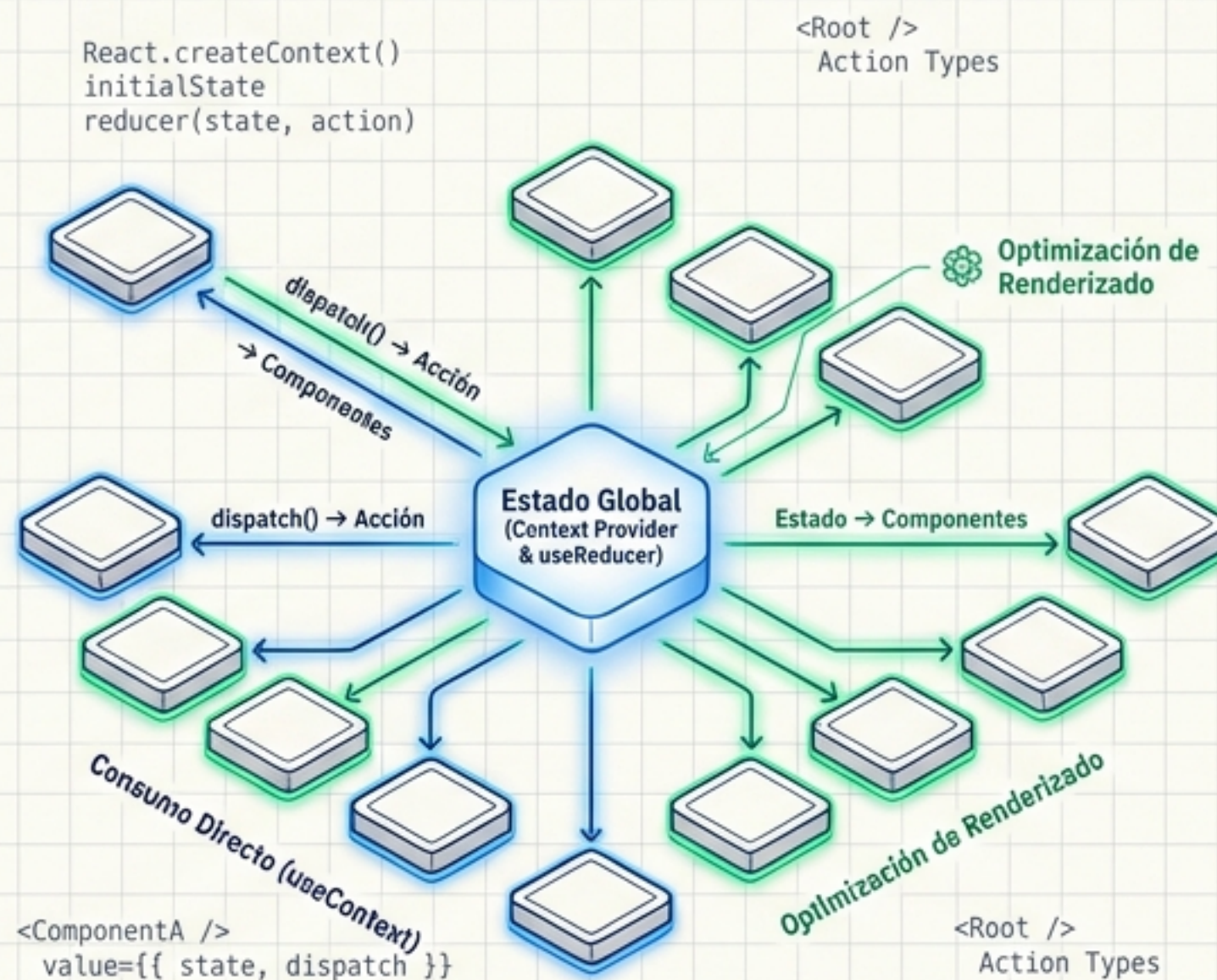


Estado Global sin Redux

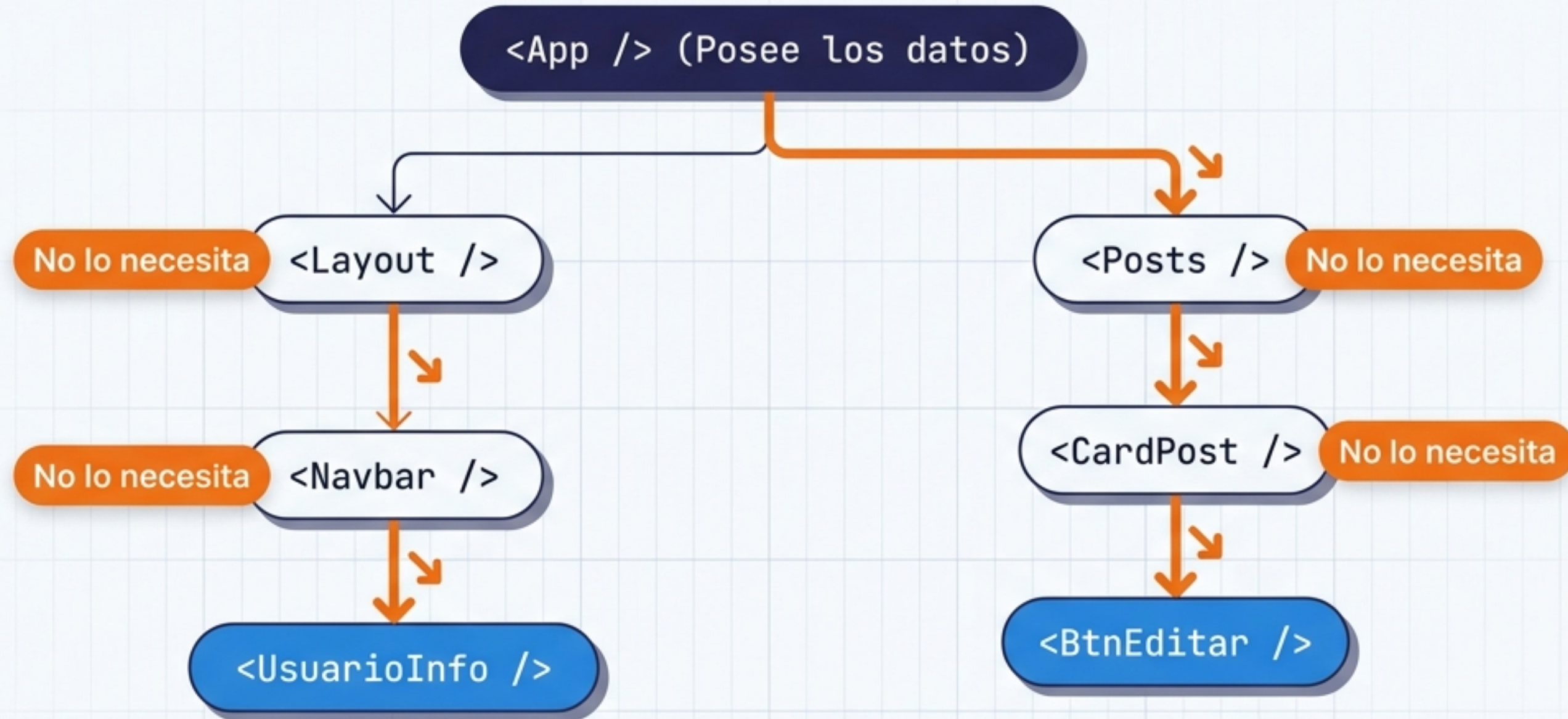
Dominando Context API y useReducer en Arquitecturas React



TRANSICIÓN
ARQUITECTÓNICA



El Problema de Arquitectura: Prop Drilling



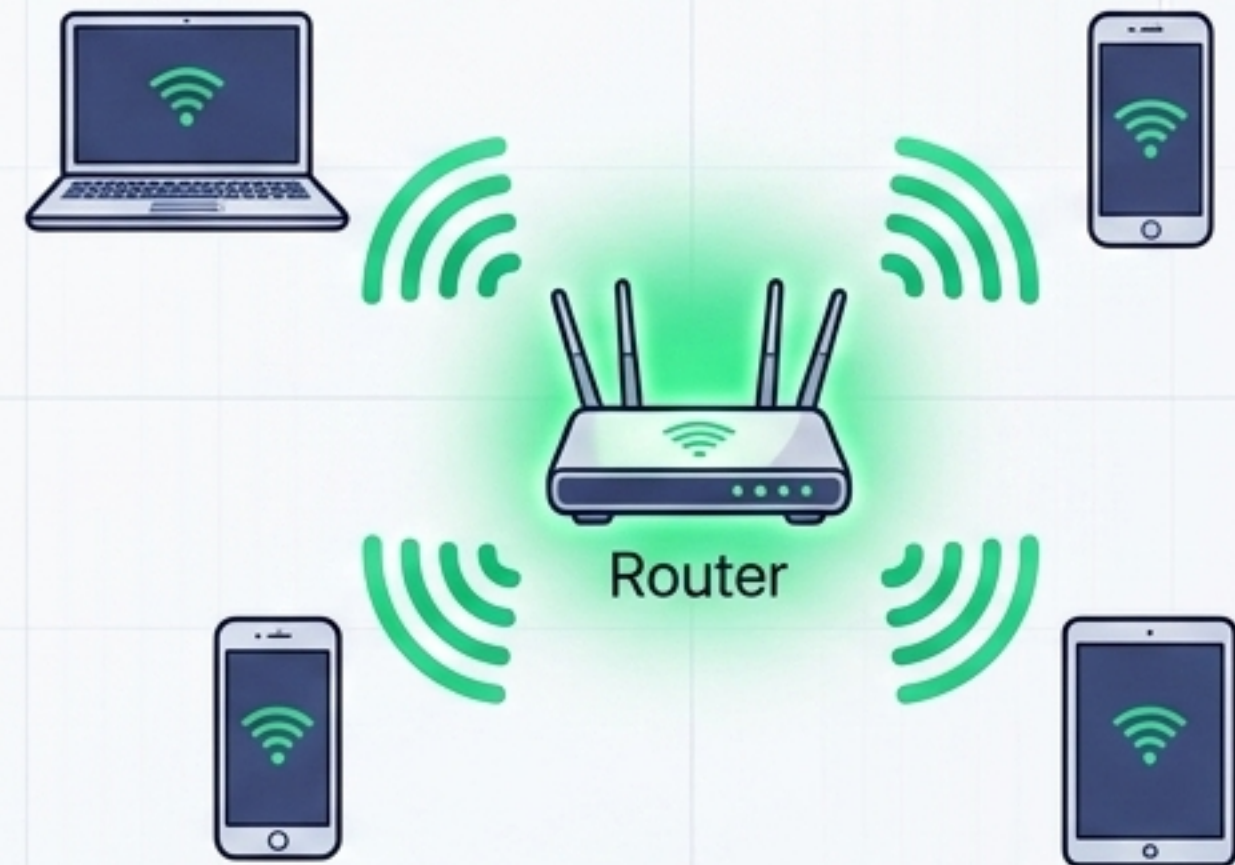
El **Prop Drilling** obliga a pasar datos por múltiples niveles intermedios que no los requieren, creando código frágil y difícil de mantener.

La Solución: El Almacén Global

El Problema: Cadena Rígida

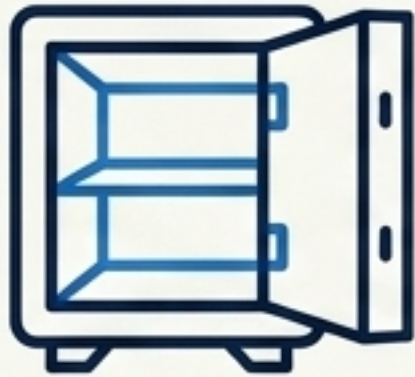


Context API: El Router WiFi



Context API resuelve el prop drilling creando un almacén global. Cualquier componente puede 'sintonizar' y acceder directamente a los datos, sin importar su profundidad en el árbol arquitectónico.

Los 3 Pilares del Context API



`createContext`

Crea el almacén vacío. Es la definición estructural donde residirá el estado global de la aplicación.



`Provider`

El transmisor. Un componente que envuelve la aplicación y distribuye activamente el estado a todos sus hijos.



`useContext`

El receptor. El hook que permite a cualquier componente aislado leer y consumir los datos del almacén global.

Paso 1: Creación del Contexto (AuthContext.jsx)

1. Instanciación del almacén vacío.

```
AuthContext.jsx

import { createContext, useState, useEffect } from 'react';

export const AuthContext = createContext(null);

export const AuthProvider = ({ children }) => {
  const [usuario, setUsuario] = useState(null);
  const [token, setToken] = useState(null);

  // Efecto para cargar sesión si hay token guardado
  useEffect(() => {
    const storedToken = localStorage.getItem('token');
    if (storedToken) {
      // Aquí iría la lógica real de validación con backend
      setToken(storedToken);
      setUsuario({ id: 1, nombre: 'Usuario Demo', rol: 'admin' });
    }
  }, []);

  const valor = {
    usuario,
    setUsuario,
    token,
    setToken
  };

  return (
    <AuthContext.Provider value={valor}>
      {children}
    </AuthContext.Provider>
  );
};
```

2. Inyección del estado local (usuario y token).

3. El objeto de valor que se emite a toda la aplicación.

El Patrón del Hook Personalizado

```
useAuth()

import { useContext } from 'react';
import { AuthContext } from '../AuthContext';

export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error("useAuth debe usarse dentro de un AuthProvider");
  }
  return context;
};
```

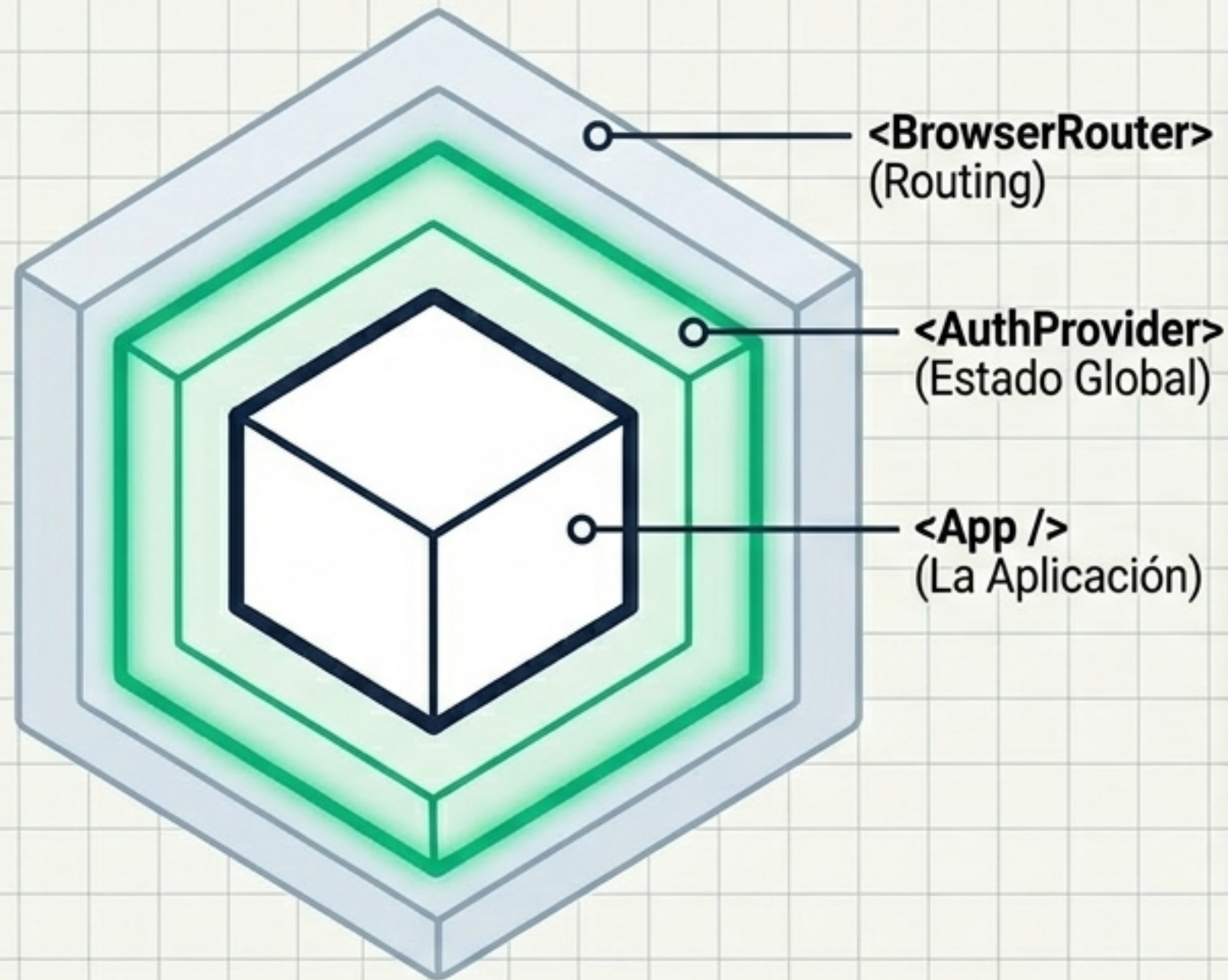
✓ Sintaxis Limpia

En lugar de importar repetidamente 'useContext' y 'AuthContext' en cada archivo, el desarrollador simplemente invoca 'useAuth()'.

✓ Seguridad Arquitectónica

Incluye manejo de errores integrado que previene fallos silenciosos, lanzando una excepción si el hook se utiliza fuera del árbol del Provider.

Paso 2: Envolviendo la Aplicación (main.jsx)



```
import { BrowserRouter } from 'react-router-dom';
import { AuthProvider } from '../context/AuthContext';
import App from './App';
// ... other imports

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <AuthProvider>
        <App />
      </AuthProvider>
    </BrowserRouter>
  </React.StrictMode>
);
```



REGLA DE ORO: El orden importa. El BrowserRouter siempre debe estar en la capa más externa, y el AuthProvider en el interior para permitir el uso de hooks de enrutamiento.

Paso 3: Consumo del Contexto sin Props

Lectura - Navbar.jsx

```
// src/components/Navbar.jsx
import { Link } from 'react-router-dom'
import { useAuth } from '../context/AuthContext'

function Navbar() {
  const { usuario, estaAutenticado, logout } = useAuth()

  return (
    <nav>
      <Link to="/">Mi Blog</Link>
      {estaAutenticado ? (
        <span>Hola, {usuario?.username}</span>
        <button onClick={logout}>Cerrar sesión</button>
      ) : (
        <Link to="/login">Iniciar sesión</Link>
      )}
    </nav>
  )
}
```

Accede al estado global directamente para renderizar condicionalmente la interfaz de usuario.

Escritura - Login.jsx

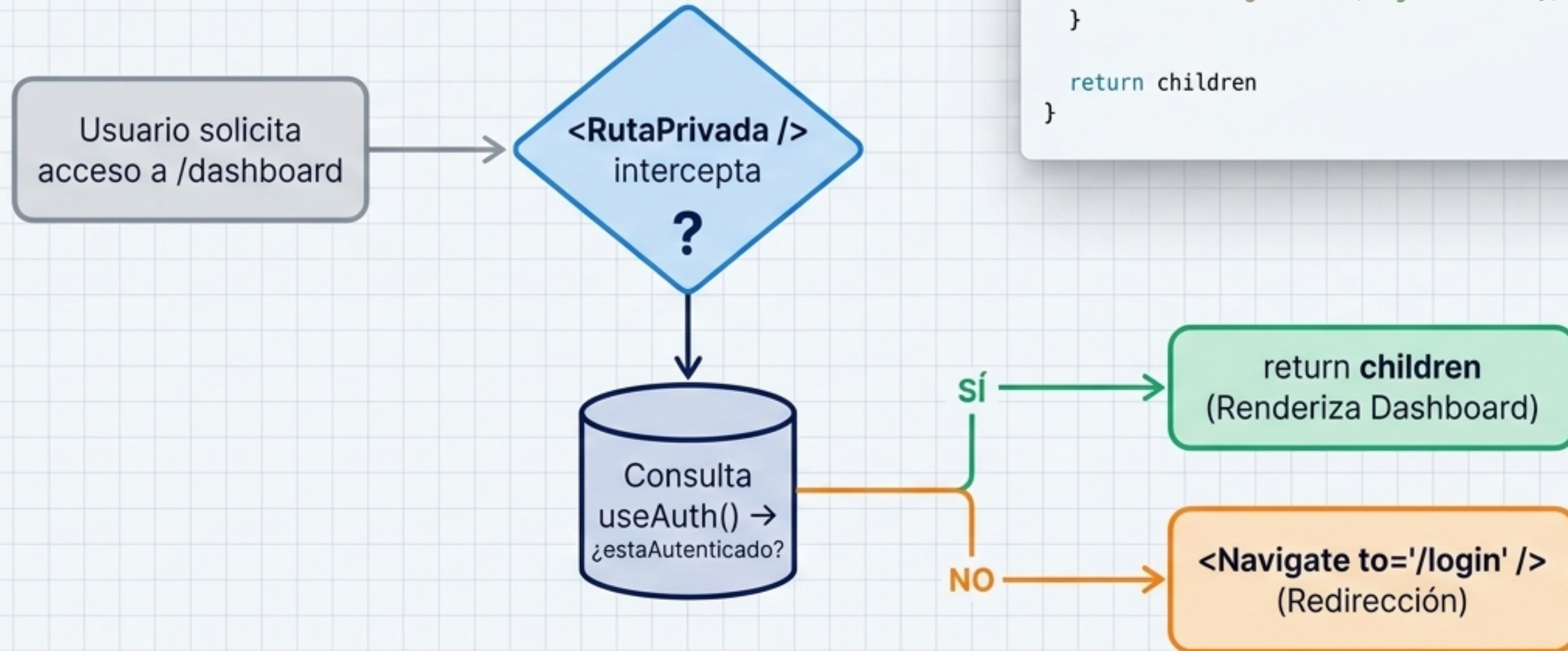
```
// src/pages/Login.jsx
import { useState } from 'react'
import { useAuth } from '../context/AuthContext'

function Login() {
  const { login } = useAuth()
  // ... state & form handling

  const manejarSubmit = async (e) => {
    e.preventDefault()
    // ... fetch logic
    try {
      // const response = await fetch(...)
      // if (response.ok) {
        login(datosUsuario, accessToken)
      }
    } catch (error) { /* ... */ }
    // ...
  }
}
```

Extrae la función mutadora e inyecta los nuevos datos al almacén, actualizando toda la app instantáneamente.

Aplicación Práctica: Protección de Rutas



```
// src/components/RutaPrivada.jsx — actualizada para usar el contexto
import { Navigate, useLocation } from 'react-router-dom'
import { useAuth } from '../context/AuthContext'

function RutaPrivada({ children }) {
  const { estaAutenticado } = useAuth()
  const location = useLocation()

  if (!estaAutenticado) {
    return <Navigate to="/login" state={{ from: location }} replace />
  }

  return children
}
```


La Evolución del Estado: **useState** vs **useReducer**

useState



- Perfecto para valores simples e independientes (booleanos, strings básicos).
- Mutación directa a través de un setter (ej. `setValor`).

useReducer



- Indispensable para estados con múltiples sub-valores interdependientes.
- Lógica centralizada, predecible y estrictamente basada en reglas matemáticas (Acciones).

Regla Arquitectónica: Escala a `useReducer` cuando el siguiente estado dependa fuertemente del estado anterior.

Anatomía de useReducer

El estado actual de la aplicación.

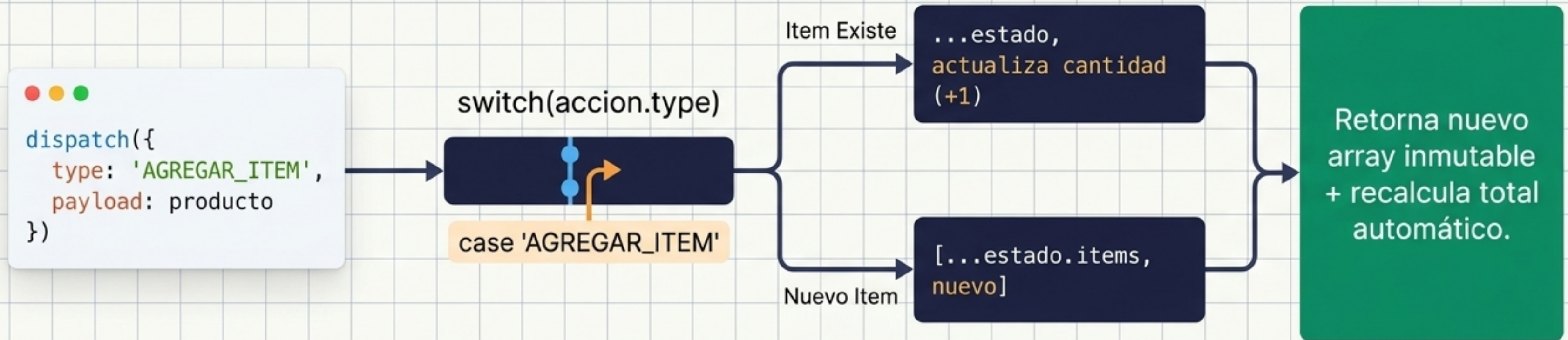
La función transmisora. Envía las acciones (**Actions**) al reducer.

```
const [estado, dispatch] =  
useReducer(reducer, estadoInicial)
```

El cerebro. La función pura que define las reglas matemáticas y lógicas de cómo cambia el estado.

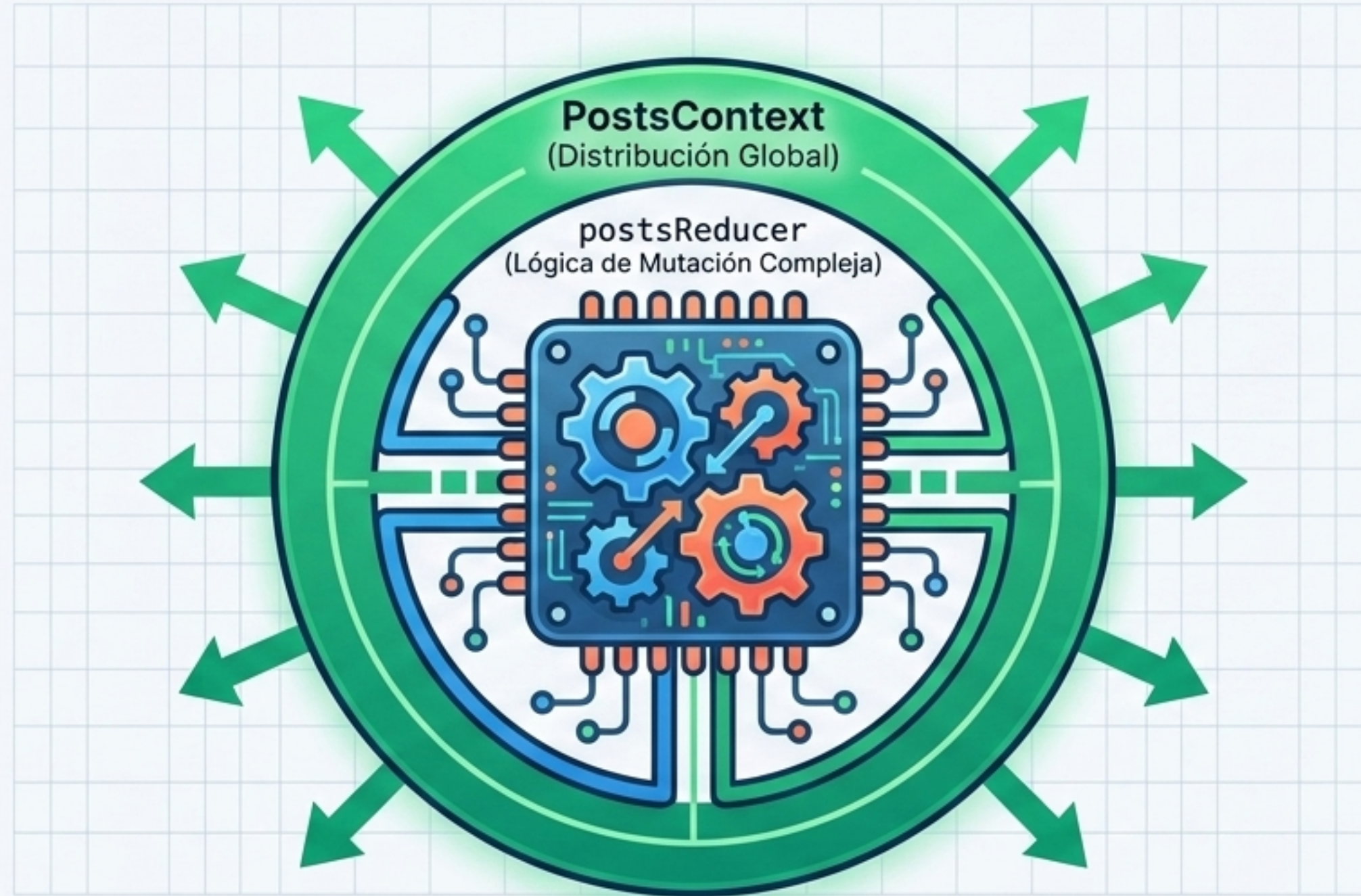
El valor o estructura base antes de que ocurra cualquier mutación.

Caso de Estudio: El Reducer del Carrito



Lógica Inmutable: El estado nunca se modifica directamente; el reducer siempre construye y devuelve un nuevo objeto de estado basado en reglas estrictas.

El Patrón Profesional: Context + useReducer



Cuando el almacén requiere **lógica de actualización compleja**, combinamos ambos hooks. Este patrón profesional nos brinda la **robustez**, **centralización** y **previsibilidad** de herramientas externas como **Redux**, pero utilizando **estrictamente herramientas nativas de React**.

Implementación de Acciones Asíncronas



```
async function cargarPosts() {  
  dispatch({ type: 'CARGANDO' })  
  try {  
    const response = await fetch(  
      'http://localhost:8000/api/posts/'  
    )  
    const data = await response.json()  
    dispatch({ type: 'POSTS_CARGADOS',  
      payload: data.results || data })  
  } catch (err) {  
    dispatch({ type: 'ERROR', payload: err.message })  
  }  
}
```

1. Activar UI de Carga

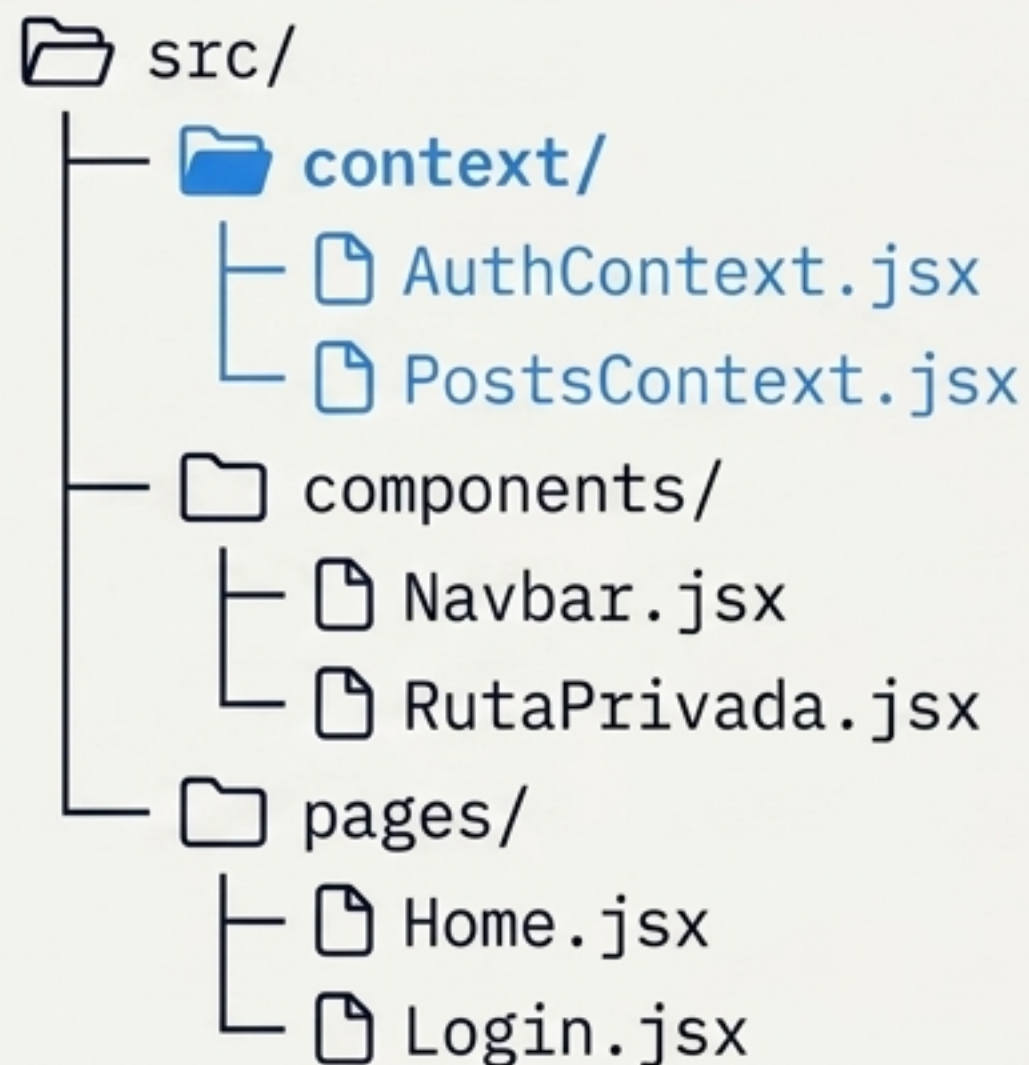
2. Operación de Red

3A. Éxito

3B. Error

Arquitectura del Proyecto y Reglas de Oro

Estructura de Directorios src/



El Manifiesto de main.jsx



Datos centralizados, lógica predecible, arquitectura escalable.